

ASSIGNMENT-6.3

NISHANTH
2303A51235

Task Description #1: Classes (Student Class)

Scenario

You are developing a simple student information management module.

Task

- Use an AI tool (GitHub Copilot / Cursor AI / Gemini) to complete a Student class.
- The class should include attributes such as name, roll number, and branch.
- Add a method `display_details()` to print student information.
- Execute the code and verify the output.
- Analyze the code generated by the AI tool for correctness and clarity.

Expected Output #1

- A Python class with a constructor (`__init__`) and a `display_details()` method.
- Sample object creation and output displayed on the console.
- Brief analysis of AI-generated code.

PROMOT:

#write a python pogram for a simple student information management system using class and methods

class Student:

```
def __init__(self, name, age, grade):  
    self.name = name  
    self.age = age  
    self.grade = grade
```

```
def display_info(self):

    print(f"Name: {self.name}, Age: {self.age}, Grade: {self.grade}")

def calculate_average_grade(students):

    if not students:

        return 0

    return sum(student.grade for student in students) / len(students)

if __name__ == "__main__":

    students = []

    num_students = int(input("Enter number of students: "))

    for i in range(num_students):

        print(f"\nStudent {i + 1}")

        name = input("Enter name: ")

        age = int(input("Enter age: "))

        grade = float(input("Enter grade: "))

        student = Student(name, age, grade)

        students.append(student)

    print("\nStudent Information:")

    for student in students:

        student.display_info()

    average_grade = calculate_average_grade(students)

    print(f"\nAverage Grade: {average_grade:.2f}")
```

OUTPUT:

```
• Enter number of students: 2

Student 1
Enter name: nish
Enter age: 20
Enter grade: 7.8

Student 2
Enter name: ishu
Enter age: 19
Enter grade: 8.5

Student Information:
Name: nish, Age: 20, Grade: 7.8
Name: ishu, Age: 19, Grade: 8.5

Average Grade: 8.15
○ PS C:\ai> █
```

Task Description #2: Loops (Multiples of a Number)

Scenario

You are writing a utility function to display multiples of a given number.

Task

- Prompt the AI tool to generate a function that prints the first 10 multiples of a given number

using a loop.

- Analyze the generated loop logic.
- Ask the AI to generate the same functionality using another controlled looping structure (e.g., while instead of for).

Expected Output #2

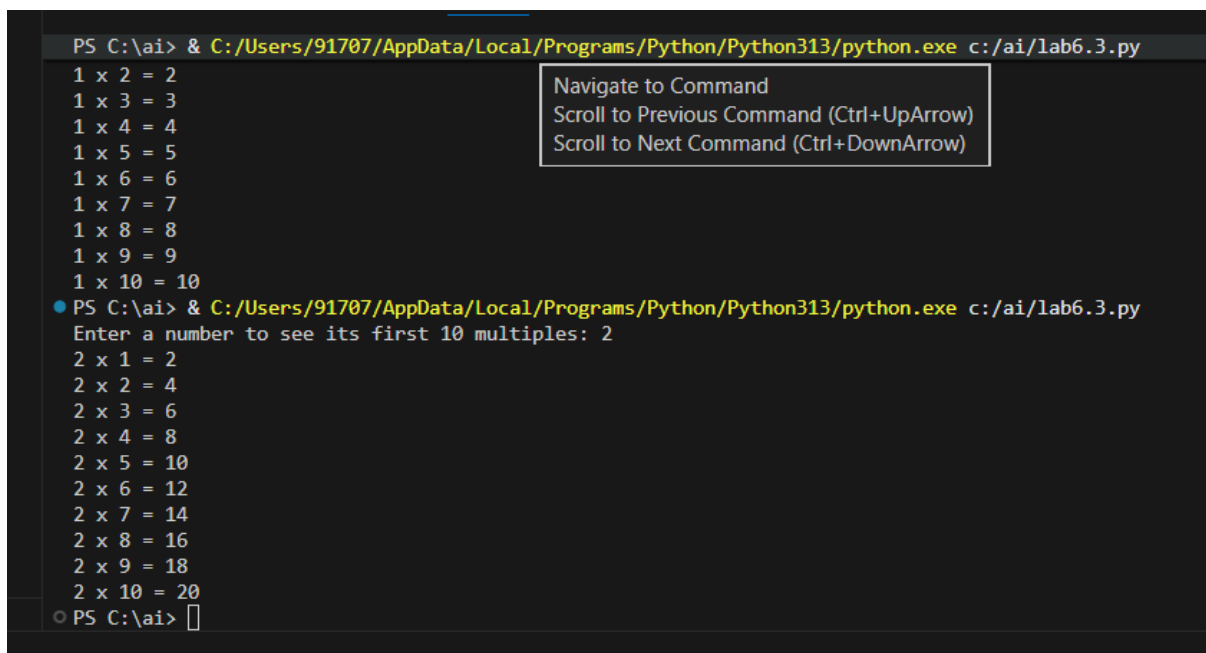
- Correct loop-based Python implementation.
- Output showing the first 10 multiples of a number.
- Comparison and analysis of different looping approaches

CODE:

#write a python program to write the first 10 multiples of the number entered by the user using loops.

```
def print_multiples(number, count=10):  
    for i in range(1, count + 1):  
        print(f"{number} x {i} = {number * i}")  
  
if __name__ == "__main__":  
    num = int(input("Enter a number to see its first 10 multiples: "))  
    print_multiples(num)
```

OUTPUT:



```
PS C:\ai> & C:/Users/91707/AppData/Local/Programs/Python/Python313/python.exe c:/ai/lab6.3.py  
1 x 2 = 2  
1 x 3 = 3  
1 x 4 = 4  
1 x 5 = 5  
1 x 6 = 6  
1 x 7 = 7  
1 x 8 = 8  
1 x 9 = 9  
1 x 10 = 10  
● PS C:\ai> & C:/Users/91707/AppData/Local/Programs/Python/Python313/python.exe c:/ai/lab6.3.py  
Enter a number to see its first 10 multiples: 2  
2 x 1 = 2  
2 x 2 = 4  
2 x 3 = 6  
2 x 4 = 8  
2 x 5 = 10  
2 x 6 = 12  
2 x 7 = 14  
2 x 8 = 16  
2 x 9 = 18  
2 x 10 = 20  
○ PS C:\ai>
```

Tooltip text:
Navigate to Command
Scroll to Previous Command (Ctrl+UpArrow)
Scroll to Next Command (Ctrl+DownArrow)

Task Description #3: Conditional Statements (Age Classification)

Scenario

You are building a basic classification system based on age.

Task

- Ask the AI tool to generate nested if-elif-else conditional statements to classify age groups (e.g., child, teenager, adult, senior).
- Analyze the generated conditions and logic.

- Ask the AI to generate the same classification using alternative conditional structures (e.g., simplified conditions or dictionary-based logic).

Expected Output #3

- A Python function that classifies age into appropriate groups.
- Clear and correct conditional logic.
- Explanation of how the conditions work.

CODE:

#write a python program of basic classification system based on age

```
def classify_age(age):  
    if age < 0:  
        return "Invalid age"  
    elif age <= 12:  
        return "Child"  
    elif age <= 19:  
        return "Teenager"  
    elif age <= 59:  
        return "Adult"  
    else:  
        return "Senior"  
  
if __name__ == "__main__":  
    age = int(input("Enter your age: "))  
    category = classify_age(age)  
    print(f"You are classified as: {category}")
```

OUTPUT:

```

PS C:\ai> & C:/Users/91707/AppData/Local/Programs/Python/Python313/python.exe c:/ai/lab6.3.py
Enter your age: 20
You are classified as: Adult
PS C:\ai> & C:/Users/91707/AppData/Local/Programs/Python/Python313/python.exe c:/ai/lab6.3.py
Enter your age: 19
You are classified as: Teenager
PS C:\ai> & C:/Users/91707/AppData/Local/Programs/Python/Python313/python.exe c:/ai/lab6.3.py
Enter your age: 30
You are classified as: Adult
PS C:\ai> 

```

Task Description #4: For and While Loops (Sum of First n Numbers)

Scenario

You need to calculate the sum of the first n natural numbers.

Task

- Use AI assistance to generate a `sum_to_n()` function using a for loop.
- Analyze the generated code.
- Ask the AI to suggest an alternative implementation using a while loop or a mathematical formula.

Expected Output #4

- Python function to compute the sum of first n numbers.
- Correct output for sample inputs.
- Explanation and comparison of different approaches.

CODE:

#write a python program to calculate the sum of the first N numers entered by the user using loops.

```
def sum_of_numbers(N):
```

```
    total = 0
```

```
    for i in range(1, N + 1):
```

```
        total += i
```

```
    return total
```

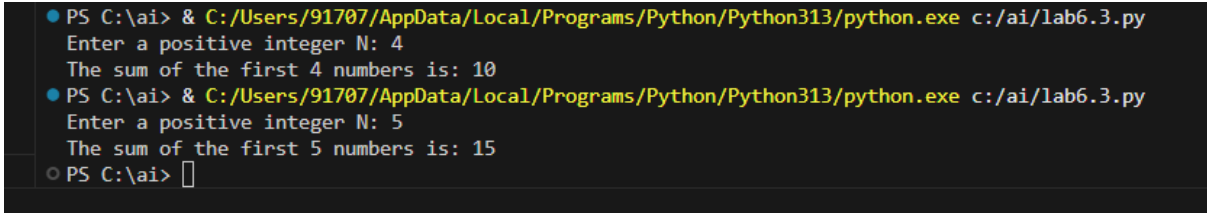
```
if __name__ == "__main__":
```

```
    N = int(input("Enter a positive integer N: "))
```

```
total_sum = sum_of_numbers(N)
```

```
print(f"The sum of the first {N} numbers is: {total_sum}")
```

OUTPUT:



```
PS C:\ai> & C:/Users/91707/AppData/Local/Programs/Python/Python313/python.exe c:/ai/lab6.3.py
Enter a positive integer N: 4
The sum of the first 4 numbers is: 10
PS C:\ai> & C:/Users/91707/AppData/Local/Programs/Python/Python313/python.exe c:/ai/lab6.3.py
Enter a positive integer N: 5
The sum of the first 5 numbers is: 15
PS C:\ai>
```

Task Description #5: Classes (Bank Account Class)

Scenario

You are designing a basic banking application.

Task

- Use AI tools to generate a Bank Account class with methods such as deposit(), withdraw(), and check_balance().
- Analyze the AI-generated class structure and logic.
- Add meaningful comments and explain the working of the code.

Expected Output #5

- Complete Python Bank Account class.
- Demonstration of deposit and withdrawal operations with updated balance.
- Well-commented code with a clear explanation.

CODE:

#Write a python program for a bank account with methods to deposit, withdraw, and check balance using classes loops and conditional statements.

```
class BankAccount:
```

```
    def __init__(self, initial_balance=0):
```

```
        self.balance = initial_balance
```

```
    def deposit(self, amount):
```

```

if amount > 0:

    self.balance += amount

    print(f"Deposited: ${amount:.2f}")

else:

    print("Deposit amount must be positive.")


def withdraw(self, amount):

    if 0 < amount <= self.balance:

        self.balance -= amount

        print(f"Withdrew: ${amount:.2f}")

    else:

        print("Insufficient funds or invalid withdrawal amount.")


def check_balance(self):

    print(f"Current balance: ${self.balance:.2f}")

if __name__ == "__main__":

    account = BankAccount()

    while True:

        print("\nBank Account Menu:")

        print("1. Deposit")

        print("2. Withdraw")

        print("3. Check Balance")

        print("4. Exit")

        choice = input("Choose an option (1-4): ")

        if choice == '1':

            amount = float(input("Enter amount to deposit: "))

```



```
        account.deposit(amount)

    elif choice == '2':

        amount = float(input("Enter amount to withdraw: "))

        account.withdraw(amount)

    elif choice == '3':

        account.check_balance()

    elif choice == '4':

        print("Exiting the program.")

        break

    else:

        print("Invalid choice. Please try again.")
```

OUTPUT:

```
PS C:\ai> & C:/Users/91707/AppData/Local/Programs/Python/Python313/python.exe c:/ai/lab6.3.py
1. Deposit
2. Withdraw
3. Check Balance
4. Exit
Choose an option (1-4): 1
Enter amount to deposit: 100
Deposited: $100.00

Bank Account Menu:
1. Deposit
2. Withdraw
3. Check Balance
4. Exit
Choose an option (1-4): 3
Current balance: $100.00

Bank Account Menu:
1. Deposit
2. Withdraw
3. Check Balance
4. Exit
Choose an option (1-4): █
```